KOD İNCELEMESİ • FORMÜLLER VE ALGORİTMALAR

Cer Gücü Simülasyonu Koddaki Formüller

Formüllerin amacı, hesaplanma biçimi ve C# kaynak kodundaki konumları

İnceleme tarihi: 8 Kasım 2026

Kapsam: Uygulamanın mevcut kaynak kodundaki hareket, direnç, kuvvet, güç, elektrik ağı ve raporlama algoritmaları.

Bu sürümde özellikle çıkarılanlar: Dış dokümanlardan alınmış parametre değerleri, araç eğrisi noktaları, hat/trafo verileri, doküman adları ve dokümanlara dayalı sayısal örnekler.

Kaynak ilkesi: Kod konumları proje kaynak koduna; genel fizik açıklamaları temel mekanik ve elektrik bağıntılarına dayanır. Bu belge herhangi bir proje dokümanındaki sayısal girdileri aktarmamaktadır.

1. Nasıl okunmalı?

Bu belge, uygulamadaki denklemlerin ve simülasyon davranışını belirleyen önemli kuralların kod incelemesidir. Kod konumları proje köküne göre verilmiştir. Değişkenler aşağıdaki gibi kullanılır:

Sembol	Anlam
`m`	Tren kütlesi
`m_eff`	Döner kütle katkısı dahil etkin kütle
`v`	Tren hızı; alt indisinde birim belirtilir
`a`	İvme
`F`, `R`	Kuvvet, direnç
`V`, `I`, `P`, `E`	Gerilim, akım, güç, enerji
`η`	Verim oranı

> **Önemli:** Buradaki açıklamalar denklem yapısını anlatır; uygulama ayarlarının mühendislik bakımından doğrulanmış olduğunu iddia etmez. Gerçek senaryo parametreleri bu belgeye dahil edilmemiştir.

Ana kaynak dosyaları

- `Services/RailwaySimulationEngine.cs`: tren hareketi ve elektrik ağı hesapları.
- `Models/SimulationModels.cs`: model özellikleri, eğri değerleme ve kuvvet limit fonksiyonları.
- `ViewModels/MainViewModel.cs`: zaman adımı ve ekrandaki özet metrikler.
- `Services/SimulationRecorder.cs`, `Models/SimulationResults.cs`: simülasyon kayıtları ve özet sonuçlar.

2. Zaman ve tren hareketi

2.1 Zaman adımı

```
dt = temel_adım × simülasyon_hız_çarpanı
```

Neden: Her zamanlayıcı vuruşunda simülasyonu ilerletmek. **Kod:** `ViewModels/MainViewModel.cs:431`. Zaman adımı sayısal çözünürlüğü etkiler; hız çarpanı da adım büyüklüğünü değiştirdiği için fiziksel zamandan ayrı bir kullanıcı ayarı değildir.

2.2 Etkin kütle ve yardımcı tüketim

```
m_eff = m_dolu + max(0,m_boş) × max(0,r_döner)/100
```

```
P_yardımcı = P_yardımcı,girdi × clamp(kullanım_oranı,0,1)
```

Neden: Döner parçaların etkisini etkin kütleye yaklaşık katmak; yardımcı yükü kullanım oranıyla ölçeklemek. **Kod:** `Services/RailwaySimulationEngine.cs:62-67`. Girdinin eksik/geçersiz olması halinde motor kodundaki yedek davranış ayrıca geçerlidir.

2.3 Birim dönüşümü ve kinematik

```
v[m/s] = v[km/h] / 3.6
Δx[m] = v[m/s] × dt[s]
Δs[km] = Δx[m] / 1000
```

```
Δv = işaret(v_hedef−v) × min(|v_hedef−v|, a_sınır×3.6×dt)
```

Neden: SI birimleriyle hareket hesabı ve sabit ivmeli zaman adımı yaklaşımı. **Kod:** `Services/RailwaySimulationEngine.cs:192-225`. İvme sınırı ile hız farkı birlikte kullanılarak bir adımda hedef hızın aşılması engellenir.

2.4 Newton'un ikinci yasasıyla çekiş ivmesi

```
a_kuvvet = max(0,(F_çekiş−R_toplam)/m_eff)
a_kullanılan = min(a_konfigüre,a_kuvvet)
```

Neden: Net kuvvetin kütleye oranı ivmeyi verir. **Kod:** `Services/RailwaySimulationEngine.cs:196-207`. Kuvvet kN ve kütle ton olduğunda oran sayısal olarak m/s^2 ile uyumludur.

Frenlemede hız azaltma ayrı olarak konfigüre edilen azami fren ivmesiyle yapılır (`RailwaySimulationEngine.cs:210-216`). Bu kinematik güncelleme elektrikli fren eğrisinden türetilmez. Dolayısıyla fren eğrisi geri kazanılabilir elektrik gücünü sınırlarken, trenin yavaşlamasını tek başına belirlememektedir.

3. Hedef hız ve tren direnci

3.1 İşletme ve güvenlik kuralları

Motor hedef hızı tren yönü, sonraki istasyon, hat hız limiti ve öndeki tren aralığına göre sınırlar. İstasyona yaklaşma için mesafeyle orantılı doğrusal hedef hız kuralı ve trenler arası mesafe için parçalı hız azaltma kuralı uygulanır.

Kod: `Services/RailwaySimulationEngine.cs:102-189`. Bunlar uygulama kontrol algoritmalarıdır; doğrulanmış fren mesafesi denklemi olarak değerlendirilmemelidir. İstasyon beklemesi, dispatch ve hat sonu dönüşü de aynı metodda işletme kurallarıdır.

3.2 Davis biçimli direnç

```
R_Davis[kN] = (A·m + B·m·v[km/h] + C·v[km/h]²) / 1000
```

Neden: Hıza bağlı mekanik ve aerodinamik tren direncini ampirik biçimde temsil etmek. **Kod:** `Services/RailwaySimulationEngine.cs:387-400`. Katsayılar model parametreleridir; bu raporda katsayı değeri veya kaynağı aktarılmamaktadır. Koddaki koşula göre geçersiz katsayı ilgili terimi sıfırlar.

3.3 Eğim direnci

```
R_eğim = m·g·(eğim_yüzdesi/100) × yön_işareti
```

Neden: Küçük eğim açısında ağırlık kuvvetinin hat doğrultusundaki bileşenini yaklaşık hesaplamak. **Kod:** `Services/RailwaySimulationEngine.cs:401-410`. İşaret, hareket yönü ve eğimin işaret tanımına göre belirlenir.

3.4 Kurp direnci

```
w_kurp = kurp_katsayı_fonksiyonu(yarıçap)
R_kurp = m·g·w_kurp / 1000
```

Kod: katsayı fonksiyonu `Models/SimulationModels.cs:35`; kuvvet uygulaması `Services/RailwaySimulationEngine.cs:411-416`. Katsayı formülünün ve sabitlerinin kaynağı bu belgede verilmemiştir. Toplam direnç `R_Davis + R_eğim + R_kurp` olarak döner.

3.5 Modelde bulunup kullanılmayan özellikler

`Models/SimulationModels.cs:21-24,36` içinde uzunluk, eğim gösterimi, yardımcı eğim kuvveti ve kurp hızı gibi özellikler bulunur. Kod çağrı zincirinde kullanılmayan özellikler, motorun aktif hesap formülü olarak yorumlanmamalıdır. `JerkLimiti` de modelde tanımlıdır; mevcut hız güncellemesi jerk sınırlaması uygulamaz.

4. Kuvvet eğrisi ve elektrik gücü

4.1 Doğrusal enterpolasyon

$$y = y_0 + (y_1-y_0)\times(x-x_0)/(x_1-x_0)$$

Kuvvet eğrisi hızla, akım limit eğrisi gerilimle enterpole edilir. Eğri aralığının dışında uç nokta değeri kullanılır. **Kod:** `Models/SimulationModels.cs:180-232`. Bu sürüm, eğri noktalarının hiçbirini listelemez veya kopyalamaz.

4.2 Eğri ve akım sınırının birleşmesi

```
P_DC,max[kW] = V[V] × I_max[A] / 1000
P_kullanılabilir = max(0,P_DC,max−P_yardımcı)
F_kullanılabilir = max(0,min(F_eğri,F_akım_limiti))
```

Neden: Kuvvet talebini trenin hız eğrisinin ve elektriksel akım kapasitesinin izin verdiği sınırda tutmak. **Kod:** `Models/SimulationModels.cs:139-178,234-262`. Kuvvet/güç dönüşümünde hız m/s kullanılır; çekiş ve rejenerasyon için verim farklı yönde uygulanır.

Düşük hız için akım-güç sınırına özel bir eşik dalı vardır; bu dal geçerliyse fonksiyon eğri kuvvetini doğrudan döndürebilir (`SimulationModels.cs:243-252`).

4.3 Kuvvet talebi

```
Hızlanma: F_talep = max(0,m_eff·max(0,a)+R_toplam)
Sabit hız: F_talep = max(0,R_toplam), tren hareket ediyorsa
Frenleme: F_fren,talep = m_eff·|a|
```

Kod: `Services/RailwaySimulationEngine.cs:420-451`. Durum sınıflandırması trenin durum metni ve ivme eşiklerini kullanır.

4.4 Mekanik ve net elektrik gücü

```
P_çekiş[kW] = F_çekiş[kN] × v[m/s] / η
P_rej[kW] = F_fren[kN] × v[m/s] × η
P_net = P_çekiş + P_yardımcı − P_rej
```

Neden: Mekanik güç, kuvvet ile hızın çarpımıdır; elektrik ve mekanik güç verimle ilişkilendirilir. **Kod:** `Services/RailwaySimulationEngine.cs:454-480`. Yardımcı güç net çekiş gücüne eklenir.

5. Elektrik ağı ve gerilim çözümü

5.1 Akım-gerilim-güç

```
I[A] = P_net[kW] × 1000 / V[V]
```

Çekişte akım pozitif, şebekeye geri beslemede negatif işaretlidir. Gerilim sıfır veya negatifse motor akımı sıfır alır. **Kod:** `Services/RailwaySimulationEngine.cs:337-340,558-560,599-601`.

5.2 Hat ve kaynak kol direnci

```
r_hat[Ω/m] = (r_katener+r_ray)[mΩ/km] / 1,000,000
R_hat = r_hat × mesafe
R_kaynak[Ω] = (R_trafo+R_fider+R_dönüş)[mΩ] / 1000
R_kol = max(ε,R_kaynak+R_hat)
```

Kod: `Services/RailwaySimulationEngine.cs:483-520`. Bu, girilen dirençlerden eşdeğer kaynak ve hat kolu oluşturan modeldir. Aday besleyici trafolar seçilirken mesafe sınırı ve devre dışı olma durumu dikkate alınır.

5.3 Trafo paylaşımı

```
G_i = 1/R_kol,_i
w_i = G_i / ΣG_j
I_TM,_i = Σ_train(I_train×w_i)
```

Neden: Daha düşük dirençli paralel besleme koluna daha yüksek pay vermek. **Kod:** `Services/RailwaySimulationEngine.cs:519-533,548-562`. Bu, ayrıntılı katener düğüm ağı çözümü değil, ağırlıklandırılmış eşdeğer-kol yaklaşımıdır.

5.4 Gerilim düşümü

```
V_TM = clamp(V_yüksüz−I_TM×R_kaynak,0,V_maks)
V_tren ≈ Σ_i w_i×(V_TM,_i−I_tren×w_i×R_hat,_i)
```

Kod: `Services/RailwaySimulationEngine.cs:573-607`. Gerilim düşümü Ohm bağıntısıyla hesaplanır. Tren akımı, gerilim ve güç birbirine bağlı olduğundan bu hesap yinelemeli yapılır (`RailwaySimulationEngine.cs:254-335`).

Yinelemede gevşetilmiş gerilim güncellemesi kullanılır ve belirli sayıda yineleme veya gerilim değişim eşiğinde durur. Bu nümerik çözüm ayarlarıdır, fizik sabiti değildir. Model; doğrultucu kontrol karakteristiği, ayrıntılı tek hat topolojisi ve toprak/kaçak akım ağını çözmez.

6. Enerji, özetler ve sınırlamalar

6.1 Adım adım enerji

```
E_tüketim[kWh] += max(0,P_net[kW])×dt[s]/3600
E_rejenerasyon[kWh] += P_rej[kW]×dt[s]/3600
```

Neden: Enerji, gücün zaman üzerindeki integralidir; motor bunu ayrık adımlarda toplar. **Kod:** `Services/RailwaySimulationEngine.cs:342-343`.

6.2 Ekran metrikleri

Canlı toplam çekiş gücü pozitif tren güçlerinin, rejenerasyon ise negatif tren güçlerinin mutlak değerlerinin toplamıdır; net güç ikisinin farkıdır. Ortalama hız/gerilim aritmetik ortalama, minimum gerilim en küçük değerdir. **Kod:** `ViewModels/MainViewModel.cs:450-468`.

6.3 Kayıt ve raporlama

Kayıtçı her snapshot için tren/trafo verilerini toplar, minimum/ortalama/azami metrikleri üretir. **Kod:** `Services/SimulationRecorder.cs:72-118`; sonuç özetleri `Models/SimulationResults.cs:67-77`.

> **Kayıt enerji hesabı uyarısı:** Kayıtçıdaki toplam enerji formülü mevcut snapshot gücünü geçen kayıt süresine yayarak çarpar. Değişken güçte bu, adım adım enerji integraliyle aynı sonuç olmak zorunda değildir. Bu rapor kaynak kodu belgeliyor; bu hesabı değiştirmiyor.

6.4 Uygunluk değerlendirmesi

Uygunluk değerlendiricisi bazı metrikler için sabit eşik karşılaştırması yapar; diğerleri doğrulanmış limit olmadan inceleme durumunda kalır. **Kod:** `Services/ComplianceEvaluator.cs:20-55`; standart kayıtları `Services/StandardsCatalog.cs`. Kodda yer alan eşik tek başına standart uygunluğu veya sertifikasyon kanıtı değildir.

Genel fizik referansları

- OpenStax, University Physics Volume 1, §3.4, sabit ivmeli hareket: openstax.org
- OpenStax, University Physics Volume 1, §5.3, Newton'un ikinci yasası: openstax.org
- OpenStax, University Physics Volume 1, §7.1, iş ve kuvvet: openstax.org
- OpenStax, College Physics 2e, §20.2, Ohm yasası: openstax.org
- OpenStax, College Physics 2e, §20.4, elektrik gücü ve enerji: openstax.org

Bu belge yalnızca kod yapısını ve genel fizik bağıntılarını açıklar; uygulama parametrelerinin kaynağını veya mühendislik geçerliliğini onaylamaz.